

Corso di Studio	INFORMATICA PER LE AZIENDE DIGITALI (L-31)
Dimensione dell'elaborato	Minimo 6.000 – Massimo 10.000 parole (<i>pari a circa Minimo 12 – Massimo 20 pagine</i>)
Formato del file da caricare in piattaforma	PDF
Nome e Cognome	Lorenzo Licalzi
Numero di matricola	0312200831
Tema n. (Indicare il numero del tema scelto):	4
Titolo del tema (Indicare il titolo del tema scelto):	Sharing technologies
Traccia del PW n. (Indicare il numero della traccia scelta):	14
Titolo della traccia (Indicare il titolo della traccia scelta):	Sviluppo di un software di geolocalizzazione culturale per condividere il patrimonio librario degli utenti privati.
Titolo dell'elaborato (Attribuire un titolo al proprio elaborato progettuale):	GeoLibrary: progettazione e sviluppo di un'applicazione web per la ricerca geospaziale di patrimoni librari privati con PostGIS e .NET Aspire

PARTE PRIMA – DESCRIZIONE DEL PROCESSO

Utilizzo delle conoscenze e abilità derivate dal percorso di studio

(Descrivere quali conoscenze e abilità apprese durante il percorso di studio sono state utilizzate per la redazione dell'elaborato, facendo eventualmente riferimento agli insegnamenti che hanno contribuito a maturarle):

Le competenze acquisite dal corso Basi di dati hanno fornito una struttura solida nel merito dell'archiviazione dati su database, in particolare la conoscenza acquisita su PostgreSQL è stata indispensabile per poter approfondire e ampliare la base di dati con le coordinate geospaziali.

Il backend del progetto è stato interamente progettato attraverso l'approccio object-oriented acquisito nel corso di Ingegneria del Software e ampliando con approfondimenti sulla Clean Architecture. Quest'ultima prevede la separazione tra Dominio, Applicazione e Infrastruttura del codice per un maggiore controllo e gestione delle modifiche.

Le informazioni degli utenti sono state gestite tenendo conto dei principi del GDPR appresi nel corso di Diritto per le Aziende Digitali: la posizione delle librerie viene mostrata in forma approssimata, l'indirizzo associato alle richieste di prestito è anonimizzato fino all'approvazione del detentore del libro ed è stata predisposta una funzionalità di cancellazione del profilo utente, in linea con il principio del diritto all'oblio.

La scelta di utilizzare Keycloak nel progetto per quanto riguarda la gestione degli utenti è basata sulle competenze apprese in Reti di Calcolatori e Cybersecurity. La gestione compliant dei dati di accesso, cancellazione, recupero password è un tema molto delicato che impatta sia il GDPR per la retention e privacy che la sicurezza dell'archiviazione del dato. Per questo motivo invece di

implementare il sistema di gestione di credenziali, sessioni e recupero password ho deciso di affidarmi a Keycloak tramite il protocollo OpenID Connect, riducendo così i rischi di una implementazione custom dei dati di accesso.

Il corso di Tecnologie Web ha fornito le basi sui protocolli e sui modelli di comunicazione su cui si fonda qualunque applicazione web moderna: il modello client-server, il protocollo HTTP/HTTPS e i formati di scambio dati come JSON. Questi concetti sono stati applicati direttamente nella progettazione delle API REST del backend, che comunicano con il client tramite richieste HTTPS scambiando dati in formato JSON. La conoscenza del funzionamento dei cookie, affrontata a lezione come meccanismo classico di gestione dello stato lato client, è stata utile anche per motivare la scelta opposta adottata nel progetto: l'autenticazione utilizza token inviati nell'header Authorization delle richieste anziché una sessione basata su cookie, soluzione più adatta a un'architettura con frontend e backend disaccoppiati.

Sempre nel corso sono state trattate le basi di HTML e CSS che sono state fondamentali per l'utilizzo del framework Vue.js per definire i componenti del Frontend. Il framework Vue.js non è stato trattato dal corso ma appreso da esperienze lavorative pregresse.

Le stesse basi di HTML sono state inoltre il punto di partenza per l'implementazione di alcuni aspetti di accessibilità dell'interfaccia, attraverso attributi aria-label su pulsanti con sola icona, alt descrittivi per le copertine dei libri e role="alert" per i messaggi critici dei form, così da permettere un utilizzo corretto anche da parte degli screen reader.

Fasi di lavoro e relativi tempi di implementazione per la predisposizione dell'elaborato

(Descrivere le attività svolte in corrispondenza di ciascuna fase di redazione dell'elaborato. Indicare il tempo dedicato alla realizzazione di ciascuna fase, le difficoltà incontrate e come sono state superate):

Fase 1: Analisi dei requisiti

Questa fase è durata circa una settimana ed è stata necessario per individuare in maniera puntuale i requisiti richiesti dalla traccia. Questo passaggio è stato propedeutico per definire le fasi di progetto e l'MVP. Le funzionalità identificate all'interno della traccia sono: Gestione del profilo e registrazione dell'utente, pubblicazione del patrimonio librario con relativi metadati e immagini di copertina, ricerca testuale e spaziale con visualizzazione su mappa, dettaglio del libro con richiesta di prestito, raccolta di statistiche, dashboard amministrativa. Dal punto di vista non funzionale le aree individuate sono tre: privacy, performance e accessibilità. Questi vincoli hanno guidato poi le scelte successive sia per quanto riguarda l'infrastruttura che per il codice. Un esempio è il vincolo delle performance sulle ricerche geospaziali per il quale ho deciso di affidarmi all'estensione geospaziale di PostgreSQL ossia PostGIS. Il vincolo di privacy invece ha anticipato decisioni come l'anonimizzazione dei dati sensibili in fase di prestito.

In questa fase è stato inoltre definito lo stack tecnologico, scelto in funzione dei requisiti individuati e dalla mia esperienza lavorativa maturata sui linguaggi e strumenti.

Questo approccio guidato dai requisiti, piuttosto che dalla familiarità con gli strumenti, ha permesso di arrivare a uno stack coerente con i vincoli di performance e privacy individuati, mantenendo comunque margini di scalabilità futura.

Fase 2: Setup del progetto

In questa fase mi sono occupato prevalentemente di impostare la soluzione Aspire. Ho definito i progetti, configurato il controllo di versione con GIT e integrato Keycloak come identity provider. L'applicazione è stata pubblicata sul mio repository pubblico di GitHub raggiungibile

tramite il link <https://github.com/krajatum/GeoLibrary>. La configurazione della solution Aspire ha permesso inoltre di verificare attraverso dati di telemetria e la dashboard integrata la corretta configurazione dell'ambiente. In questa fase ho anche iniziato a configurare il progetto .NET Api con una struttura Clean Architecture andando a separare le logiche di dominio dalla parte infrastrutturale. Questo ha reso necessario un refactoring di tempistiche contenute del primo prototipo scritto di servizio.

Fase 3: Geolocalizzazione e PostGIS

Mi sono occupato di configurare tutto quello che concerne la geolocalizzazione. Questo include l'integrazione con OpenStreetMap per la gestione degli indirizzi, la colonna PostGIS per i dati geospaziali, l'aggiunta del pacchetto NetTopologySuite in Entity Framework per il supporto ai dati spaziali e l'implementazione dei primi prototipi di metodi di ricerca per raggio e per poligono. La gestione di PostGIS e la sua integrazione con Entity Framework è stata la cosa che ha preso più tempo poiché la prima implementazione non usava correttamente le coordinate geospaziali ed è stato necessario cambiare il tipo di dato del campo in "geometry". La ricerca dell'indirizzo testuale è stata lasciata parzialmente incompleta, da completare in fasi successive. Il tempo di questa fase è di circa tre giorni.

Fase 4: Modello librerie, libri e basi Frontend

Completata la configurazione di PostGIS, sono passato alla definizione dei modelli e delle entità per le librerie e le statistiche, e ho iniziato a configurare la parte di frontend. In un primo momento avevo scelto la libreria Tailwind, poi sostituita con Vuetify per una maggiore stabilità e velocità di prototipazione. Ho accettato questo tradeoff tra flessibilità e semplicità soprattutto per una questione di manutenibilità del codice, considerando che Vuetify è una libreria matura, con una community ampia e un ciclo di rilasci regolare. Ho fatto uso anche della libreria keycloak-js che mi ha permesso di integrare l'autenticazione con Keycloak con semplicità e sicurezza.

Fase 5: Gestione libri, immagini e ricerca

In questa fase ho implementato la parte più corposa del progetto ossia l'inserimento dei libri, l'importazione semplificata tramite un file csv di ISBN, la schermata di dettaglio e l'integrazione con di MinIO e SkiaSharp per la gestione delle immagini di copertina. La scelta di utilizzare MinIO è stata dettata dalla necessità di avere un objectstorage, ossia un software di archiviazione dei documenti, che ha anche il vantaggio di essere compatibile con altri objectstorage in cloud come AWS S3 o Azure Blob Storage. Questa compatibilità permette di avere la flessibilità di deploy e soprattutto di scalabilità rispetto ad una soluzione fatta in casa.

L'importazione tramite ISBN si basa su <https://openlibrary.org/api/books>, un sito che permette di ricercare attraverso una chiamata http l'isbn e recuperare un JSON con le informazioni di titolo, autore, editore, data di pubblicazione, descrizione, URL della copertina e i "subjects" (usati poi per abbinare le categorie interne).

Fase 6: Privacy, prestiti, statistiche e categorie

Ho introdotto l'anonimizzazione dell'indirizzo nelle richieste di prestito per garantire la privacy, implementato il flusso completo di richiesta, visione e restituzione del libro e realizzato il tracciamento delle visite sui singoli libri e librerie lato backend. In un secondo momento ho introdotto degli endpoint per l'esposizione delle statistiche, modifica del profilo e dell'avatar dell'utente e l'aggiunta delle categorie, prima mancanti, per i libri sia in fase di inserimento manuale che in fase di importazione tramite file.

Fase 7: Dashboard amministrativa e rifiniture

Nell'ultima fase ho realizzato la dashboard amministrativa, sostituendo il placeholder iniziale dell'endpoint con l'implementazione reale che aggrega i dati raccolti attraverso il tracciamento introdotto nella Fase 6. Per questa implementazione è stato necessario aggiungere un ruolo admin su Keycloak per proteggere l'accesso alla sezione e realizzato sul frontend una pagina dedicata con i grafici dell'andamento nel tempo delle visualizzazioni di libri e librerie, e la classifica dei libri più visitati.

Risorse e strumenti impiegati

(Descrivere quali risorse - bibliografia, banche dati, ecc. - e strumenti - software, modelli teorici, ecc. - sono stati individuati ed utilizzati per la redazione dell'elaborato. Descrivere, inoltre, i motivi che hanno orientato la scelta delle risorse e degli strumenti, la modalità di individuazione e reperimento delle risorse e degli strumenti, le eventuali difficoltà affrontate nell'individuazione e nell'utilizzo di risorse e strumenti ed il modo in cui sono state superate):

Il fulcro centrale del progetto è .NET 10 con il supporto dell'orchestrazione di Aspire. Aspire è un prodotto di Microsoft che permette di replicare in locale un sistema composto da più componenti (API, database, Keycloak, MinIO) che può essere poi rilasciato molto agevolmente su ambienti cloud. Mette a disposizione inoltre una dashboard integrata che permette di tracciare lo stato dell'ambiente, gli output raw dei container, i log strutturati e soprattutto le tracce OpenTelemetry. Quest'ultime sono risultate indispensabili in fase di debug dell'applicativo per identificare eventuali errori nascosti e anche le performance delle chiamate.

La parte del frontend è stata implementata in Vue 3, un framework JavaScript modulare per semplificare la scrittura dei componenti. La scelta è dovuta alle esperienze che ho maturato nelle esperienze lavorative precedenti, con l'aggiunta dello studio delle differenze Vue 2 e Vue 3 sul quale avevo un'esperienza molto limitata. In aggiunta ho introdotto Vuetify, una libreria di componenti per Vue 3, che contiene molti componenti utili alla creazione dell'interfaccia e soprattutto un supporto a lungo termine e standardizzato. Da segnalare l'attenzione sull'accessibilità da parte della libreria. Per la mappa ho utilizzato Leaflet e OpenStreetMap, quest'ultimo è un provider gratuito di mappe, alternativa al classico Google Maps. La scelta di OpenStreetMap è stata dettata anche da considerazioni costi e della gestione delle chiavi che andrebbero ruotate per mantenere l'applicativo in sicurezza.

Ho scelto di utilizzare PostgreSQL + PostGIS per l'archiviazione dati. Questa scelta è dettata dalla necessità di implementare delle ricerche attraverso dati spaziali e, dopo diversi approfondimenti online e il background acquisito dal corso di Basi di Dati, mi è sembrata la scelta più corretta. Al fine di integrare .NET 10 ed Entity Framework Core, un ORM di Microsoft per .NET, ho aggiunto la libreria NetTopologySuite che aggiungeva il supporto ai campi nativi di PostGIS.

Come accennato nei capitoli precedenti, ho scelto di utilizzare l'identity provider Keycloak per delegare la complessità dell'autenticazione e per la conformità GDPR. La possibilità di configurare ed esportare il realm inoltre permette anche la replica semplificata dell'ambiente. Inoltre è possibile andare a configurare tutti gli aspetti di autenticazione dell'utente e anche la visualizzazione nativa di login, rendendolo un prodotto completo anche per l'utilizzo enterprise.

MinIO è un Object Storage compatibile con AWS S3 che permette l'archiviazione delle copertine

di libri, librerie e gli avatar degli utenti. Ho scelto questo software per rendere il progetto compatibile con altre soluzioni cloud e quindi scalabile in futuro, senza però appesantire troppo lo sviluppo locale. L'accesso alle immagini non avviene tramite bucket pubblico, ma attraverso URL firmati con validità temporanea, generati dinamicamente dal backend: questo evita di esporre pubblicamente e in modo permanente i contenuti caricati dagli utenti.

Per creare le copertine di libri e librerie è stato necessario introdurre anche una libreria .NET per l'elaborazione delle miniature. La libreria scelta è SkiaSharp, una libreria con licenza MIT, quindi ad uso libero per prodotti commerciali e non commerciali.

Ho scelto di utilizzare OpenLibrary Books API per arricchire i metadati dei libri in fase di import massivo. Ho accettato il tradeoff sui dati non sempre perfetti o standardizzati anche per l'assenza di costi rispetto ai suoi concorrenti come ISBNdb API. Questo però ha comportato l'implementazione di sovrastrutture per il recupero dei dati dal JSON e il parsing dei dati.

Infine per gestire il versioning del progetto e tenere traccia delle modifiche effettuate ho utilizzato Git e pubblicato il progetto sul provider GitHub per renderlo pubblico. La pubblicazione su GitHub risponde al requisito esplicito della traccia di un repository pubblico; prima della pubblicazione ho verificato che nei file di configurazione versionati (variabili d'ambiente, export del realm Keycloak) non fossero presenti credenziali o segreti in chiaro. Sono stati aggiunti successivamente gli utenti di esempio al fine di predisporre un ambiente locale riproducibile senza sforzi, inevitabilmente con delle password di esempio in chiaro.

PARTE SECONDA – PREDISPOSIZIONE DELL'ELABORATO

Obiettivi del progetto

(Descrivere gli obiettivi raggiunti dall'elaborato, indicando in che modo esso risponde a quanto richiesto dalla traccia):

Il primo obiettivo del progetto è implementare un applicativo che supporti la pubblicazione e ricerca del patrimonio librario privato. L'utente registrato può creare la sua libreria impostando la posizione attraverso la geolocalizzazione dell'indirizzo, popolarla di libri con metadati completi (titolo, autore, ISBN, anno, descrizione, categorie) e renderla ricercabile da altri utenti tramite ricerca testuale o spaziale. È il nucleo funzionale attorno a cui è stato costruito tutto il resto.

Il secondo obiettivo definito dalla traccia era applicare principi di UX e accessibilità. Sono stati adottati attributi ARIA sui form e pulsanti icona, testo alternativo descrittivo delle copertine e feedback visivo delle azioni (snackbar, stati di caricamento).

Il terzo obiettivo, gestire dati di geolocalizzazione e filtri spaziali, è stato raggiunto attraverso PostGIS e NetTopologySuite: la ricerca supporta sia un raggio attorno a un punto sia un'area disegnata liberamente sulla mappa, con le query spaziali eseguite direttamente a livello di database anziché in memoria applicativa, scelta necessaria per mantenere prestazioni accettabili al crescere del numero di librerie.

Il quarto obiettivo, la gestione delle immagini e relative anteprime, è coperto sia per le copertine dei libri, le immagini delle librerie e per l'immagine del profilo dell'utente. Il caricamento di un'immagine genera automaticamente una miniatura e l'accesso ai file avviene tramite URL temporaneo firmato, non esponendo così pubblicamente l'immagine.

Raccogliere e presentare statistiche d'uso era il quinto obiettivo del progetto. Questo obiettivo ha necessitato di una particolare attenzione alle regolamentazioni GDPR e alla performance nella gestione dei dati. Le statistiche vengono presentate ai proprietari della libreria e del libro, e poi aggregate in una dashboard amministrativa per soli amministratori del sito. La dashboard amministrativa include grafici ed è stata gestita correttamente l'accessibilità dei dati agli screen reader.

L'ultimo obiettivo era implementare e rispettare le regolamentazioni per la privacy e consenso dei dati personali. Questo ha richiesto l'introduzione di un'informativa sul trattamento dei dati personali e di un banner per il consenso esplicito al tracciamento delle visite, la cui scelta viene salvata localmente nel browser e può essere revocata in qualsiasi momento, oltre alla possibilità di eliminare definitivamente il proprio account e tutti i contenuti associati.

Contestualizzazione

(Descrivere il contesto teorico e quello applicativo dell'elaborato realizzato):

GeoLibrary vuole inserirsi in un contesto sempre crescente di sostenibilità e sharing economy, valorizzando così le risorse private che rimangono spesso inutilizzate negli scaffali e riducendo gli sprechi di una economia che spinge sempre verso un nuovo acquisto. Iniziative come il book crossing mettono alla luce una sempre più ampia richiesta per questo tipo di servizio, solitamente limitato ad alcune aree geografiche. Normalmente le attività di book sharing vengono gestite da archiviazioni sovvenzionate come ad esempio il Sistema Bibliotecario dei Castelli Romani (<https://www.bibliotechecastelliromani.it/>) che permettono agli utenti di prendere in prestito libri dalle biblioteche che partecipano a questo consorzio. GeoLibrary nasce per creare uno strumento di condivisione che non si limita ad un'area geografica ristretta e permette anche ai privati cittadini che non hanno a disposizione un catalogo già definito del loro patrimonio di poter condividere e censire i loro libri.

Questo servizio si pone nel panorama dello sharing per i privati cittadini e piccole associazioni culturali prive di una propria infrastruttura digitale, ampliando così l'economia di condivisione.

Il servizio non si vuole limitare ad un'area geografica specifica ma più a una comunità specifica di persone che condividono lo stesso interesse: condividere il proprio patrimonio librario privato. Questa scelta ha avuto delle ripercussioni sulle scelte architetturali dell'applicativo, in particolare la geolocalizzazione non serve a delimitare l'accesso ad una specifica area geografica ma invece limitare quella che potrebbe essere una ricerca su un'area geografica indefinita. Questo permette all'applicativo di essere utilizzabile potenzialmente anche da utenti che nelle loro vicinanze non hanno librerie a loro disposizione, ricercando così libri anche a chilometri di distanza.

Come già accennato precedentemente gli utenti target si racchiudono in due categorie principali. La prima categoria sono i cittadini privati che possiedono una libreria personale, anche semplicemente uno scaffale, e che invece di lasciarlo a prendere polvere scelgono di condividerla e rendere così i libri disponibili alla consultazione. La seconda categoria sono le piccole associazioni culturali che gestiscono un fondo librario ma non dispongono di una soluzione digitale per il catalogo e i prestiti. Per loro GeoLibrary diventa una struttura a costo nullo, che assolve alle stesse funzioni dell'OPAC Bibliotecario. L'OPAC Bibliotecario infatti è un catalogo digitale delle biblioteche del Servizio Bibliotecario Nazionale, nel quale però non confluiscono, come già accennato, i piccoli fondi librari privati. Questo permette all'associazione inoltre di non

dover implementare personalmente un prodotto e gestire poi l'infrastruttura privatamente, riducendo così i costi.

Un elemento che rafforza la sostenibilità di questo applicativo, per quanto non implementato, è la possibilità futura di rendere la piattaforma economicamente autosufficiente. Questo sarebbe possibile introducendo un sistema di donazioni volontarie integrato direttamente nella piattaforma, ad esempio utilizzando Stripe Connect. Stripe Connect è il prodotto di Stripe pensato apposta per le piattaforme che mettono in relazione due parti, in questo caso il proprietario del libro e il donatore. GeoLibrary come intermediario tratterebbe una parte minima della donazione come costo di gestione e il resto verrebbe donato alla libreria. Questo permetterà di evitare un ente finanziatore esterno (un comune, un'università, un'associazione), rendendo GeoLibrary indipendente e rispecchiando così la direzione non istituzionale e distribuita del progetto con gli stessi utenti che renderebbero sostenibile nel tempo l'infrastruttura che utilizzano. Per i principi di trasparenza sarà necessario predisporre una grafica chiara e esaustiva di come viene frazionata la donazione.

Descrizione dei principali aspetti progettuali
(Sviluppare l'elaborato richiesto dalla traccia prescelta):

Alla base del progetto c'è l'idea di rendere il prodotto scalabile e espandibile facilmente, utilizzando tecnologie moderne che ne aiutano lo sviluppo e la gestione. Per questo motivo ho scelto di affidarmi ad Aspire, un prodotto di Microsoft pensato proprio per replicare architetture complesse sul proprio ambiente locale di sviluppo. Questo ha permesso di individuare facilmente gli errori, integrare servizi disaccoppiati tra di loro con facilità e inoltre di utilizzare le tecnologie più congeniali al progetto grazie al suo supporto a multipli linguaggi di programmazione. L'utilizzo di Aspire inoltre permetterebbe l'implementazione futura di ulteriori microservizi con compiti di dominio, come ad esempio la gestione delle donazioni menzionata precedentemente, che potrebbe essere integrata molto facilmente nell'ambiente creato.

Il cuore pulsante del sistema è il servizio backend scritto in C# utilizzando il framework .NET 10 pubblicato da Microsoft che ormai da anni rientra tra i principali prodotti per la creazione di applicazioni web. Nativamente il framework è stato progettato per funzionare su tutti i sistemi operativi da Windows a Linux e offre un supporto alla containerizzazione estremamente ben documentato. L'ecosistema di pacchetti, installabili attraverso il package manager NuGet, è estremamente ben fornito e rende quindi il processo di sviluppo e manutenzione dell'applicativo facile e lineare.

Al backend ho affiancato un frontend moderno, una SPA (Single Page Application) che è stata scritta in Vue3. Vue è un framework JavaScript, ormai con anni di maturità, che fornisce strumenti e funzionalità per creare un'applicazione Web resiliente e sicura. Il vantaggio di utilizzare un framework come Vue è la possibilità di creare componenti isolati che accettano in input dei dati ed emettono degli eventi in output, rendendo così il frontend un'applicazione modulare, facile da mantenere e debuggare.

L'ecosistema di librerie di Vue permette di accedere a funzionalità che avrebbero richiesto molto tempo di implementazione, come la libreria di componenti Vuetify che mette a disposizione componenti resilienti e sicuri. La libreria utilizza le linee guida di Material Design definite da

Google, aggiungendo la possibilità di estendere il tema con dei semplici file di configurazione.

I prodotti di terze parti integrati nel progetto svolgono compiti isolati e specifici, volti ad avere una separazione netta delle responsabilità. Keycloak ad esempio si occupa in toto della gestione, sicurezza e manutenzione del modulo autenticazione, fornendo una dashboard completa a supporto. Il vantaggio di usare Keycloak come identity provider è inoltre delegare tutte le problematiche di implementazione di un'autenticazione custom ad un prodotto che è uno standard di settore. Fornisce inoltre delle funzionalità come OpenID Connect, di cui ho fatto uso per l'autenticazione, che sarebbero state complesse da implementare e mantenere soprattutto gli standard di sicurezza e privacy richiesti dall'autenticazione.

L'altro prodotto utilizzato è MinIO, un object storage compatibile con AWS S3, che permette di caricare e scaricare documenti e immagini tramite chiamate HTTP. La compatibilità con S3 amplia le possibilità future, permettendo di passare a sistemi cloud in maniera trasparente. Uno storage separato divide inoltre i dati relazionali da quelli binari, permettendo di alleggerire il db operativo dai file in formato binario.

Alla base di tutto il sistema c'è il database PostgreSQL con l'aggiunta dell'estensione PostGIS per il supporto ai dati geospaziali.

Lo schema relazionale è organizzato attorno a cinque entità principali: User, Library, Book, Category e LoanRequest, a cui si affiancano le aggregazioni giornaliere usate per le statistiche ma è la gestione della componente geografica, a caratterizzare le scelte di progettazione più significative del database. Un approccio ingenuo sarebbe stato rappresentare le coordinate geospaziali con due colonne Latitudine e Longitudine e confrontare poi le coordinate direttamente sull'applicativo. Questo approccio funzionerebbe molto bene su pochi record ma all'aumentare del volume dei dati per calcolare un'area o una distanza sarebbe stato necessario andare a scorrere uno per uno tutti i record. Per questo motivo ho usato il tipo di dato geometry(Point, 4326), introdotto da PostGIS di PostgreSQL, che permette di definire una colonna secondo lo standard SRID 4326 ossia un punto identificato attraverso latitudine e longitudine. Ho successivamente integrato all'interno del backend, attraverso la libreria NetTopologySuite, il campo sull'entità Library. La libreria ha permesso di indicizzare il campo con un indice spaziale GiST (Generalized Search Tree), aumentando così le performance nel recupero dei dati attraverso delle query spaziali.

Entrando nella gestione utenti, la tabella User è stata introdotta al fine di ampliare l'utente creato su Keycloak, rappresentato da un Guid, con delle informazioni dell'avatar, email e displayName. L'utente può modificare i suoi dati, tranne la mail (univoca per realm su Keycloak), attraverso un'apposita pagina di profilo sull'applicativo. Tutte le tabelle collegate all'utente sono in CASCADE, permettendo così di implementare in maniera semplice l'eliminazione dei dati legati all'utente per il diritto all'oblio del GDPR. La scelta di Keycloak inoltre permette di utilizzare OpenID Connect, un Identity Layer costruito sopra il protocollo OAuth 2.0, che permette di verificare l'identità dell'utente e fornire alcune informazioni base del profilo. Inoltre, per quanto non utilizzato nell'attuale versione dell'applicativo, è possibile collegare provider di identità esterni come Google e Facebook.

Tutte le immagini degli avatar e delle copertine di libri e librerie sono salvate sull'object storage dopo essere state ridimensionate per garantire delle performance ottimali quando vengono

richiamate dal frontend. Per fare questo è stata utilizzata la libreria opensource SkiaSharp che mette a disposizione delle utility per definire anche il tipo di campionamento in fase di ridimensionamento, garantendo così che l'immagine rimanga di qualità simile all'originale. Ogni immagine viene richiamata sempre con un URL firmato temporaneo per recuperarla senza esporre direttamente il bucket dell'object storage. Questo garantisce che qualora il libro, libreria o utente associato all'immagine vengono cancellati, non sia possibile accedere ad eventuali immagini residue. L'URL firmato inoltre garantisce che dopo un'ora sia necessario richiedere un nuovo URL per visualizzare l'immagine.

Una volta definito come i contenuti (libri, librerie e profili) vengono salvati, il passaggio successivo è la lettura di questi dati. La ricerca spaziale descritta in precedenza, per raggio o per poligono, non è l'unico modo messo a disposizione. La ricerca utilizzando un filtro testuale, applicato agli stessi risultati, permette di restringere la ricerca per titolo, autore o codice ISBN. Il confronto avviene tramite corrispondenza parziale e senza distinzione di maiuscole e minuscole utilizzando la funzione ILike di PostgreSQL, riuscendo così a trovare un risultato anche solo scrivendo una parte del titolo del libro o del nome dell'autore.

Una volta trovato il libro che si stava cercando è possibile poi, accedendo alla scheda del libro, richiederlo in prestito. Questo innesca un flusso di prestito dove viene inserita la richiesta che è visibile sia dal detentore del libro che dal richiedente, ed è visibile direttamente sulla pagina del libro oppure su una pagina dedicata dove vengono riepilogati tutti i prestiti e richieste di prestito attuali e passate. Il flusso di prestito si articola con una semplice macchina a stati dove lo stato iniziale è Richiesto, poi il detentore del libro può decidere di rifiutare e passare nello stato finale Rifiutato oppure approvarlo e lo stato passa in Approvato. Da Approvato poi può passare in Riconsegnato quando il proprietario clicca l'apposito pulsante. Un dettaglio importante è che l'indirizzo del proprietario non è visibile fino all'approvazione della richiesta di prestito.

Un punto centrale del progetto è la gestione delle statistiche relative alle visite delle librerie e dei libri. Questa funzionalità ha richiesto un'analisi approfondita al fine di garantire un accesso performante ai dati salvati, mantenendo la granularità necessaria per poter effettuare delle analisi corrette. Sono state implementate due tabelle BookDailyView e LibraryDailyView che contengono il dato aggregato per giorno delle visualizzazioni e soprattutto non riportando informazioni legate all'utente viene garantito anche l'anonimato richiesto dal GDPR. Ogni utente, una volta dato il consenso al trattamento della privacy sull'apposito banner nel frontend, viene identificato da una stringa chiamata UserSignature creata solo lato Frontend e salvata nel localStorage del browser che viene passata quando viene effettuata una visita. L'utilizzo di Redis ha garantito l'unicità della visita per libreria o libro, andando a scrivere la chiave contenente la UserSignature e i riferimenti a libreria o libro con una scadenza a mezzanotte dello stesso giorno. In questo modo ogni utente, a meno di utilizzare diversi browser o la cancellazione dei dati dello stesso, concorrerà a una sola visita giornaliera, fornendo così dei dati attendibili per il proprietario della libreria o del libro e dell'amministratore del sistema. Inoltre per garantire delle performance ho previsto un campo denormalizzato ViewsCount sul singolo libro in modo da non dover aggregare i dati ogni singola volta.

L'amministratore di sistema, definito con l'apposito ruolo Admin su Keycloak, può infatti visualizzare dei dati statistici del completo sistema attraverso la dashboard dedicata, con dati aggregati delle visite negli ultimi sette, trenta e novanta giorni oltre ad una tabella riepilogativa

dei libri più visitati. Questo garantisce una vista complessiva del sistema, sul suo andamento e permette delle analisi sulle decisioni di sviluppo future. Tutte le tabelle e grafici sono stati pensati con un'attenzione principale all'accessibilità il che ha richiesto delle attività aggiuntive proprio per garantire l'accesso agli screen reader. Infatti la libreria chart.js, implementata da vue-chartjs, non permette agli screen reader di funzionare correttamente. Per questo motivo sono stati aggiunti degli attributi aria-label riassuntivi e per i grafici delle tabelle dati alternative apribili da tastiera per gli utenti che utilizzano uno screen reader. Questo esempio è visualizzabile all'interno della dashboard amministrativa.

Il punto finale, ma non per questo meno importante, è la gestione della privacy e del consenso. Seguendo le disposizioni del GDPR, ho fornito un'informativa sul trattamento dei dati personali definendo chiaramente i dati trattati e il loro utilizzo. Viene specificato in maniera chiara come vengono gestiti i dati dell'account, dati legati al profilo, la gestione delle posizioni delle librerie pubblicate dall'utente, le statistiche e la garanzia di anonimato, e la gestione delle richieste di prestito. Inoltre sono elencati tutti i sistemi esterni utilizzati dall'applicativo come Nominatim/OpenStreetMap e OpenLibrary. Inoltre è riportato chiaramente cosa significa prestare il consenso al trattamento dei dati ossia l'abilitazione del campo per il tracciamento delle visite. Infatti, come anticipato precedentemente, il campo X-User-Signature viene generato esclusivamente lato frontend e non contiene alcun dato riconducibile all'utente. Il backend lo riceve nell'header della richiesta e lo utilizza soltanto come componente della chiave temporanea su Redis per riconoscere le visite già conteggiate nella giornata, senza mai salvarlo in una tabella del database né associarlo all'identità dell'utente: allo scadere della chiave, a mezzanotte, non ne resta traccia. Il campo inoltre viene aggiunto solamente quando viene prestato il consenso al trattamento dei dati personali attraverso l'apposito banner. Inoltre è possibile revocare il consenso e può essere fatto dall'apposito pulsante nella pagina del trattamento dei dati personali prima descritta.

Un altro punto legato alla gestione della privacy è l'offuscamento della posizione della libreria in fase di ricerca del libro. Il risultato della ricerca non riporta la posizione esatta della libreria ma una posizione approssimativa deterministica in modo che l'utente finale non possa intuire la posizione reale attraverso multiple ricerche. La posizione della libreria viene fornita solamente quando il proprietario del libro, a fronte di una richiesta di prestito, approva il prestito.

Come descritto inoltre nel paragrafo legato alla gestione delle immagini, l'utilizzo di URL firmati garantisce che le immagini non risultino pubbliche, aggiungendo un'ulteriore tassello alla gestione della privacy.

Campi di applicazione

(Descrivere gli ambiti di applicazione dell'elaborato progettuale e i vantaggi derivanti della sua applicazione):

Se la contestualizzazione ha definito il tipo di comunità a cui GeoLibrary si rivolge, resta da chiarire in quali situazioni concrete questo progetto si colloca, e con quali vantaggi rispetto alle alternative esistenti.

Il primo è il pubblico più grande ossia collezionisti privati e famiglie che hanno una raccolta di libri e vogliono renderla accessibile. Un esempio di utilizzatore potrebbe essere i collezionisti di fumetti e manga che avrebbero la possibilità di rendere disponibili alcuni numeri della loro collezione che non sono attualmente in ristampa e risultano introvabili sui mercati online. In questo modo si creerebbero anche delle comunità tematiche che incentiverebbero ancora di più l'utilizzo del book sharing.

Le piccole associazioni culturali invece potrebbero utilizzare il GeoLibrary per avere un catalogo molto semplificato del loro fondo librario senza dover adottare un prodotto complesso. Questo gli permetterebbe di avere un'infrastruttura informatica praticamente gratuita, scegliendo anche cosa voler esporre all'esterno e cosa invece permettere di condividere con tutti gli utenti.

Per esempio un circolo di lettura all'interno di un caffè letterario, che gestisce le richieste di prestito utilizzando un Excel condiviso, potrebbe semplificare le richieste di prestito e tenere un catalogo aggiornato censendo i propri libri all'interno della piattaforma.

Un altro utilizzatore molto comune potrebbe essere gli studenti universitari e delle superiori.

Potrebbero usare GeoLibrary per prendere in prestito libri universitari o per degli esami, incoraggiando la cultura del book sharing e della sostenibilità che rimane un argomento molto discusso anche dalle testate giornalistiche soprattutto all'inizio del percorso di studio o dell'anno scolastico.

GeoLibrary potrebbe poi, come accennato precedentemente, diventare indipendente implementando un sistema di donazioni tra donatore e detentore del libro. Questo sistema incentiverebbe gli utenti a mettere in condivisione i propri libri al fine di ricevere delle donazioni spontanee dagli altri utenti. GeoLibrary si sosterrrebbe prendendo una percentuale di questa donazione, specificando la quota in maniera trasparente in fase di donazione. Questo genererebbe un circolo virtuoso incentivato dalla community stessa, rendendo il book sharing un qualcosa che può aiutare anche piccoli fondi librari a mantenere la propria collezione.

In tutti questi scenari si può notare come GeoLibrary non vuole andare a sostituire il catalogo digitale delle biblioteche del Servizio Bibliotecario Nazionale ma vuole andare a riempire lo spazio che lascia scoperto, quello del patrimonio librario privato e informale che normalmente viene tralasciato dai sistemi tradizionali.

Valutazione dei risultati

(Descrivere le potenzialità e i limiti ai quali i risultati dell'elaborato sono potenzialmente esposti):

La valutazione dei risultati raggiunti da GeoLibrary non può limitarsi a un elenco di funzionalità implementate, ma deve confrontarsi direttamente con gli obiettivi dichiarati dalla traccia e con quanto affermato precedentemente. Accanto ai risultati che considero completamente raggiunti dall'applicativo e verificabili nel codice pubblicato su GitHub, esistono delle funzionalità rimaste parziali o non affrontate delle quali voglio comunque parlare. Questa autovalutazione vuole essere un'analisi dell'applicativo oggettiva, rendendo questo passaggio utile ad analizzare non solamente i punti positivi raggiunti dal progetto ma anche i margini di miglioramento. La completezza di una funzionalità è stata quindi definita su base oggettiva: viene considerata raggiunta solo se effettivamente implementata e funzionante, non se semplicemente abbozzata o prevista durante lo sviluppo.

Il risultato più solido, ed anche il fulcro dell'intero progetto, riguarda la ricerca geospaziale che non è rimasta allo stadio di prototipo ma funziona già direttamente sul database. I parametri di ricerca inviati dall'utente vengono convertiti in predicati spaziali eseguiti direttamente da PostGIS. Nella ricerca per raggio la funzione `IsWithinDistance`, valutata sul tipo di dato `geography`, calcola una distanza sferica espressa in metri e seleziona le librerie che ricadono entro il raggio indicato; nella ricerca per area, invece, il filtro verifica il contenimento del punto all'interno del poligono disegnato sulla mappa. In entrambi i casi l'interrogazione si appoggia all'indice spaziale GiST descritto in precedenza. Il vantaggio reale di questo approccio è che nessuna elaborazione viene fatta in memoria dall'applicativo su ogni record ma il database scarta a priori le righe che non soddisfano i requisiti spaziali. Questo ha un impatto notevole sulle prestazioni di ricerca al crescere del numero dei record salvati sul database in quanto

un'elaborazione sull'applicativo avrebbe richiesto di leggere tutti i record e calcolare la distanza ad ogni richiesta.

Altrettanto completo è il ciclo di vita dei contenuti multimediali. Il caricamento di un'immagine, che si tratti di una copertina, di una libreria o di un avatar, genera automaticamente una miniatura con lato lungo ridotto a 400 pixel tramite SkiaSharp, utilizzando un ricampionamento cubico Mitchell scelto per evitare le scalettature tipiche delle riduzioni marcate. Il fattore di scala non supera mai il valore di uno così da evitare di ingrandire e ridurre la qualità di immagini caricate più piccole. Un altro aspetto di cui vado particolarmente fiero, sia per la gestione dell'immagine stessa sia per la tutela della privacy, è il recupero dell'immagine attraverso un URL firmato con scadenza a un'ora. In questo modo il bucket dell'object storage non viene mai esposto direttamente, garantendo inoltre che dopo la cancellazione di un libro, di una libreria o di un utente non sia più possibile ottenere l'accesso a eventuali immagini residue.

Un altro risultato raggiunto del quale ritengo molto importante parlare è la gestione delle statistiche, il quale ha richiesto una estensiva analisi per raggiungere gli obiettivi di performance richiesti. In particolare l'aggregazione transazionale giornaliera delle statistiche e la denormalizzazione del campo ViewsCount sull'entità del libro sono le soluzioni adottate per evitare che la crescita del volume dei dati diventasse con il passare del tempo un collo di bottiglia. Questi due sistemi permettono di creare statistiche rapidamente, come ad esempio le visite di una libreria negli ultimi trenta giorni, riducendo anche il tempo di computazione richiesto da questa attività.

Inoltre la dashboard dell'amministratore è stata progettata per essere un cruscotto complessivo del sistema con l'accessibilità al primo posto.

Collegandomi al tema dell'accessibilità, è stato un problema affrontato in corso d'opera che ha richiesto una buona parte di tempo per raggiungere dei risultati soddisfacenti. Uno dei principali problemi da risolvere è stata la mancanza di funzionalità di accessibilità dei grafici creati utilizzando la libreria chart.js. Per aggirare questo problema ho dovuto inserire degli attributi aria-label e trovare una soluzione alternativa per rendere accessibili i dati agli screen reader. Dopo diverse ricerche su delle modalità alternative ho optato per creare una tabella con i dati che vanno a concorrere alla creazione del grafico che fosse espandibile attraverso la tastiera. Risolto questo grande ostacolo sono stati aggiunti anche degli attributi alt per le immagini e il lang sulla pagina HTML.

Ho infine affrontato il problema della privacy, un obiettivo molto importante per il progetto. Come discusso precedentemente il codice è stato pensato con la privacy al primo posto, salvando solo i dati necessari per il funzionamento dell'applicativo e dove non necessario evitare di tracciare il comportamento dell'utente. Questo è particolarmente visibile dalla gestione del consenso per il tracciamento delle statistiche dove l'utente può prestare il consenso, negarlo e revocarlo attraverso la pagina di privacy. Inoltre anche nel tracciamento delle statistiche, l'utente che ha generato la visita non viene mai esposto sul database poiché la stringa che identifica il visitatore viene generata lato frontend, non contiene alcun dato riconducibile all'utente e non viene mai salvata nel database, ma utilizzata soltanto come chiave temporanea per evitare visite duplicate nella stessa giornata. L'informativa della privacy riporta per intero la funzionalità e le informazioni che vengono salvati dal sistema, i servizi esterni e la finalità dei dati salvati. Inoltre un altro punto sul quale è stata richiesta molta attenzione è la posizione delle librerie: nei risultati

di ricerca l'indirizzo esatto non viene mai esposto, ma sostituito da una posizione approssimata in modo deterministico, così che ricerche ripetute restituiscano sempre lo stesso valore e non permettano di identificare l'indirizzo reale.

Dopo aver affrontato quelli che sono gli obiettivi raggiunti dal progetto bisogna però anche parlare dei limiti di quest'ultimo. Una funzionalità incompleta riguarda la ricerca per categoria. Per quanto la funzionalità in fase di censimento manuale del libro o dell'importazione tramite ISBN permette l'inserimento della categoria, la funzione di ricerca non permette di filtrare i risultati per categoria. L'impatto di questa modifica sarebbe contenuto poiché riguarderebbe una piccola modifica all'endpoint di ricerca e un redesign del componente frontend per permettere la scelta.

Sul fronte della qualità del codice, la quantità di test automatici è rimasta minima predisponendo il progetto di test e una pipeline di integrazione. In un progetto di questa entità che potrà essere in futuro esteso la mancanza di una copertura di test estensiva espone a possibili errori di regressione e la verifica delle funzionalità sarà necessario effettuarla manualmente. La scelta è stata determinata dal tempo disponibile in quanto mi sono voluto concentrare sulle funzionalità richieste dalla traccia, risultando però una delle criticità che dovranno essere risolte con priorità se il progetto dovesse proseguire.

Infine, per quanto riguarda l'accessibilità, tutte le funzionalità descritte precedentemente sono state guidate dall'esperienza diretta e dalla documentazione reperita online. Manca quindi un audit da un sistema automatizzato che certifichi la correttezza del lavoro svolto. Le soluzioni trovate sono sicuramente risolutive dei problemi affrontati ma non posso garantire che siano conformi con gli standard di accessibilità.

Nessuno di questi limiti compromette l'utilizzo dell'applicativo ma non menzionarli avrebbe reso la valutazione parziale in quanto anche riconoscere i limiti del lavoro svolto è propedeutico alla crescita dell'applicativo e alla crescita personale nella gestione dei progetti.